

Национальный исследовательский университет ИТМО
Факультет программной инженерии и компьютерной техники
Дисциплина «Разработка мобильных приложений»

Мобильное приложение для трейдинга и инвестиций

Работу выполнили:

Покалюхин И. И. · Нестеров В. А. · Джохадзе А. Б. · Бондаренко А. А. · Лашкул А. В.

Санкт-Петербург 2026

Актуальность

- Более **30 млн** частных инвесторов на Московской бирже; **> 70 %** сделок на рынке акций — физические лица.
- Основной канал доступа — **мобильные приложения**: смартфон фактически заменил биржевой терминал.
- Жёсткие требования: котировки **в реальном времени**, **корректные и атомарные** сделки, **тысячи одновременных** клиентов в пик волатильности.

Цель и задачи

Цель — спроектировать и реализовать экосистему биржевого терминала, обслуживающую **10 000 одновременных клиентов**.

Задачи:

- драйвер-источник котировок в ядре Linux;
- сбор котировок: брокер сообщений + история;
- сервис БД: счета, ордера, сделки, матчинг лимиток;
- gateway: аутентификация, WebSocket, проксирование;
- Android-приложение (Jetpack Compose);
- нагрузочное тестирование на 10 000 клиентов и наблюдаемость.

Т3: назначение и заинтересованные лица

Назначение. Доступ частного инвестора к биржевым торгам со смартфона: котировки, история, заявки, портфель, история операций.

Заинтересованные лица (основа для требований):

- частный инвестор (пользователь)
- брокерская компания (заказчик)
- поставщик котировок (биржа)
- разработчики mobile / backend
- инженер эксплуатации (DevOps/SRE)
- ИБ и комплаенс
- тестировщик (QA)
- служба поддержки

Т3: функции и состав системы

Функции

- регистрация и аутентификация
- котировки в реальном времени
- история-свечи (1m...1d)
- заявки Market / Limit
- матчинг лимиток
- портфель и P&L, история сделок

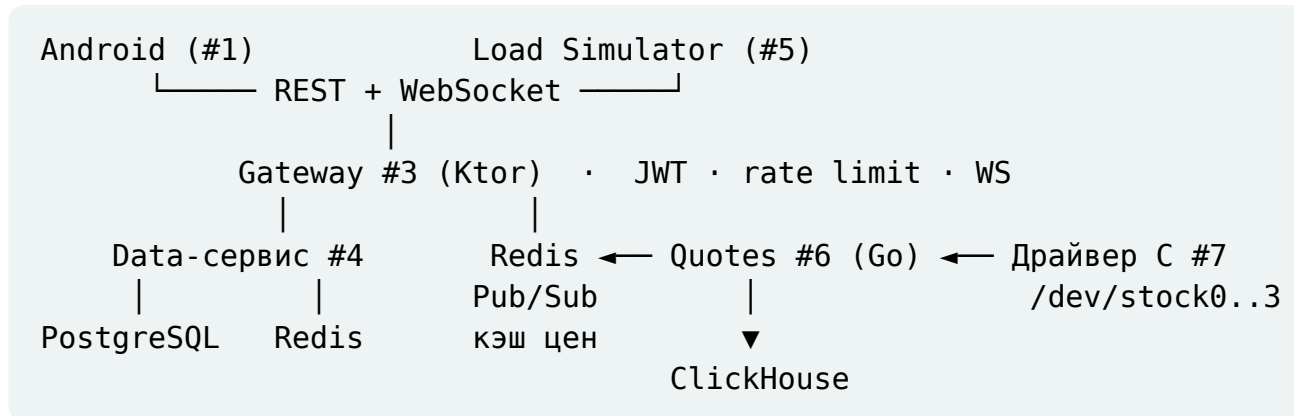
Состав (7 компонентов + инфраструктура)

- #7 драйвер · #6 quotes
- #4 data-сервис · #3 gateway
- #1 Android · #2 RN (опц.)
- #5 нагрузочный имитатор
- Redis · PostgreSQL · ClickHouse
- OpenTelemetry / Jaeger · Docker

ТЗ: характеристики системы

- **Производительность:** до 10 000 клиентов, целевой p99 $\leq$ 250 мс.
- **Надёжность:** доля ошибок $< 0,1$ %; развязка через брокер; атомарные торговые транзакции.
- **Безопасность:** JWT, bcrypt, только параметризованный SQL, rate limit.
- **Энергопотребление (мобильный аспект):** push по одному WebSocket вместо опроса; перерисовка только изменившихся элементов.
- **Инструментальный аспект:** единый язык Kotlin, Docker, OpenTelemetry.

Архитектура системы



Котировки: драйвер → quotes → Redis / ClickHouse → Gateway → клиенты.

Торговля: клиент → Gateway (JWT) → Data-сервис — сделка в одной транзакции.

Архитектура: технологии и данные

- **Языки и платформы:** C — драйвер; Go — quotes и нагрузочный имитатор; Kotlin + Ktor — backend; Jetpack Compose — Android.
- **Хранилища:** Redis — шина тиков и кэш цен; PostgreSQL — счета и сделки (транзакции); ClickHouse — история тиков и свечи.
- **База данных:** users, accounts, instruments, orders, trades, positions; В ClickHouse — ticks и материализованные candles_1m.
- **Принцип:** слабая связанность, единая точка входа (gateway), голый SQL без ORM.

Реализация: источник котировок (#7, #6)

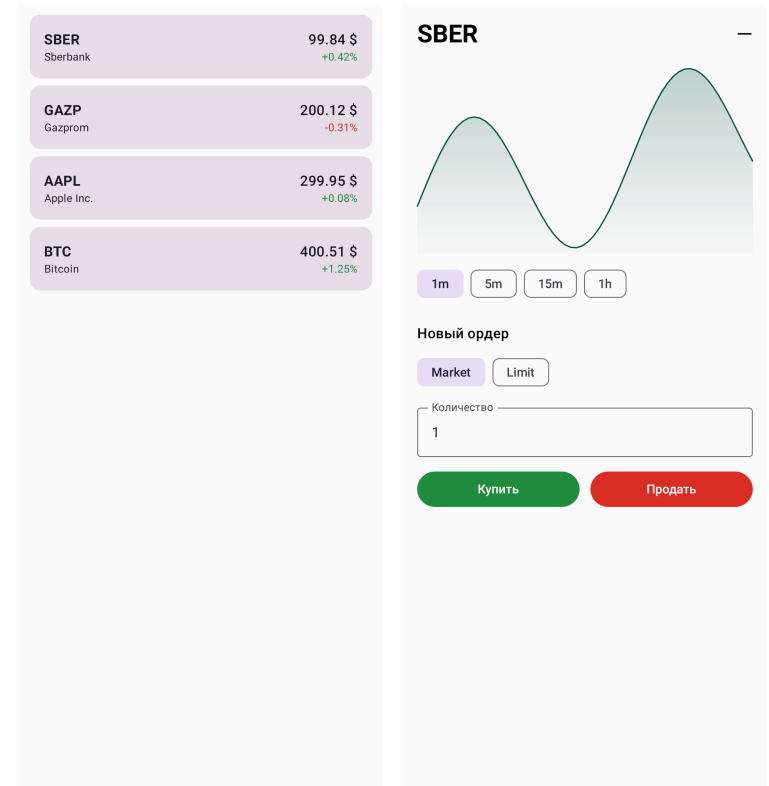
- **Драйвер ядра Linux:** символьные устройства `/dev/stock0..3`, цена по модели геометрического броуновского движения, запись 32 байта, настройка через `ioctl`.
- **Сервис quotes (Go):** читает тики → публикует в Redis Pub/Sub + батч-запись истории в ClickHouse; метрики в формате Prometheus.
- **Отказоустойчивость:** встроенный GBM-симулятор, когда устройства недоступны (например, не-Linux окружение).

Реализация: бэкенд и торговля (#4, #3)

- **Data-сервис:** голый SQL + HikariCP. Ордер выполняется атомарно: SELECT ... FOR UPDATE → проверка средств → запись сделки → пересчёт позиции и баланса.
- **Лимитные заявки:** фоновый воркер по потоку тиков (FOR UPDATE SKIP LOCKED); идемпотентность по `clientId`.
- **Gateway:** bcrypt + JWT, WebSocket fan-out (одна подписка на Redis на процесс), rate limiting, проксирование с подстановкой `userId` из токена.

Реализация: Android-приложение

- Kotlin + Jetpack Compose, **MVVM** (ViewModel + StateFlow).
- Экраны: рынок (живые цены по WebSocket), карточка инструмента с графиком и формой ордера, портфель с P&L, история заявок и сделок.
- JWT в DataStore; **авто-refresh** токена при 401 и повтор запроса.



Реализация: ключевые технические решения

- **Redis Pub/Sub как шина тиков** — развязка генерации котировок и потребителей; отказ одного потребителя не влияет на других.
- **Денежные поля строками в JSON** — сохранение точности NUMERIC.
- **Сквозной трейсинг** — W3C traceparent (OpenTelemetry), спаны SQL — дети клиентского запроса.
- **Защита от перегрузки** — rate limiting + неблокирующие буферы тиков.

Тестирование: корректность

- **Юнит-тесты** бизнес-логики: маппинг символов, средняя цена позиции, пересечение лимиток, JWT, перцентили.
- **Скриншот-тесты UI** без эмулятора (Robolectric Native Graphics + Roborazzi) — экраны рендерятся в CI.
- **Интеграционный сценарий** на реальном окружении: регистрация → market-ордер → сделка → портфель; лимитка исполняется воркером; идемпотентность повторного запроса.

Тестирование: нагрузка 10 000 клиентов

Действие	Запросов	p99, мс	Ошибок
quotes	110 119	120	0
portfolio	49 257	123	0
order	36 987	144	5
trades	24 543	120	0
history	24 277	209	5
register	10 000	122	0

- Всего **255 183** запроса, **10** ошибок (**0,004 %**).
- $p99 \leq$ **210 мс** по всем операциям.
- WebSocket: **1 000** соединений, **6,08 млн** тиков, **0** переподключений.
- Цель «10 000 клиентов» **подтверждена**.

Заключение

- Реализована **полная экосистема** биржевого терминала: драйвер → quotes → data-сервис → gateway → Android-приложение.
- Цель достигнута: **10 000 клиентов**, p99 ≤ 210 мс, доля ошибок 0,004 %.
- Обеспечена **наблюдаемость**: сквозные трейсы, метрики, структурированные логи.
- **Направления развития**: частичное исполнение ордеров, стакан заявок и матчинг «клиент против клиента», горизонтальное масштабирование Gateway.

Спасибо за внимание!

Вопросы?

Мобильное приложение для трейдинга и инвестиций · ИТМО, 2026